blink[well] — how releases reach users

For stakeholders — how one git push turns into a live website, installers, and a Python package. No jargon required.

THE BIG PICTURE

Everything starts from **one GitHub repository**. Two kinds of pushes fan out to three destinations — automatically, with no manual steps after the push:

git push
to the `main` branch → **Netlify**
website deploys (~1 min)

git push of a tag
e.g. `v0.1.2` → **GitHub Actions**
cloud "runners" build & publish → **GitHub Release + PyPI**
4 platform installers + `uvx` package

GITHUB → NETLIFY: THE WEBSITE

Continuous deployment, watched by Netlify itself

Netlify is connected to the repository through the GitHub app. It watches the `main` branch: **every push triggers a new deploy** — Netlify pulls the latest code, reads our `netlify.toml` config, and serves the `site/` folder at `blinkwell.netlify.app`. No build step (the page is plain HTML/CSS/JS), so a deploy takes about a minute. There's nothing to click and no server to babysit — the "sync" is just Netlify listening for pushes.

The site also hosts the install scripts and maps short URLs (`/i`, `/i.ps1`) to them via a `_redirects` file, so users copy one short command.

GITHUB → GITHUB RELEASE + PYPI: THE PRODUCT

Pushing a version tag (like `v0.1.2`) is the **release trigger**. It fires two independent workflows on GitHub Actions — Actions being GitHub's automation service where clean, temporary cloud machines ("runners") execute our build recipes:

Workflow 1 — `release.yml` → GitHub Release

Spins up **4 runners in parallel** — Linux, macOS (Apple Silicon), Windows x64, Windows ARM64 — each compiling the Rust binary natively for its platform. The results are attached to a GitHub Release for that tag. Because the site's installers download from `releases/latest/download/…`, they automatically serve whatever the newest release contains — no site changes needed.

Workflow 2 — `publish-pypi.yml` → PyPI

Builds the Python package with `uv` and uploads it to PyPI, the Python package index. Once live, `uvx blinkwell` works on any OS — this is also the path for platforms we don't ship binaries for (Intel Macs,

ARM Linux).

HOW PYPI KNOWS THE UPLOAD IS REALLY US — "TRUSTED PUBLISHING"

There is **no API key or password stored anywhere**. Instead, PyPI is configured once to trust *this specific workflow file in this specific repository*. At upload time:

1. The GitHub runner asks GitHub for a short-lived identity token — cryptographic proof of *"I am publish-pypi.yml, running for santpandey/eye-strain-detection, on tag v0.1.2"*.
2. It presents that token to PyPI (OIDC — an open standard for this exchange).
3. PyPI verifies the token against the trusted-publisher configuration and accepts the upload. The token expires in minutes and can't be reused or stolen.

This is the modern recommended mechanism: nothing long-lived to leak, rotate, or commit by accident — a key part of why the pipeline is low-maintenance.

WHY A TAG, AND WHY IT CAN'T BE MOVED AFTERWARDS

- **A tag is a sealed snapshot** — it pins the exact code a release is built from. Commits landing on `main` later don't leak into it.
- **PyPI is immutable** — once `blinkwell 0.1.2` exists it can never be replaced (a deliberate supply-chain protection). That's why every release starts by bumping the version number in both manifests, then tags.
- **Reproducible builds** — runners are fresh machines every time, so binaries don't depend on anyone's laptop state.

WHAT A RELEASE LOOKS LIKE, END TO END

1. Bump versions	`0.1.3` in the Python + Rust manifests, lockfile synced.
2. Verify	Format, lint, all tests green (Rust + Python).
3. Push main	GitHub stores the code; Netlify redeploys the site automatically.
4. Push the tag	Fires both workflows in parallel.
5. Done (~10 min)	4 installers on the Release page + new version on PyPI. Install commands on the site keep working — they always fetch "latest".

Takeaway: the team pushes code and a tag — everything else (website, installers for 4 platforms, the Python package) is automation that runs in the cloud, with no stored credentials and no manual uploads.